



AI Nutritionist

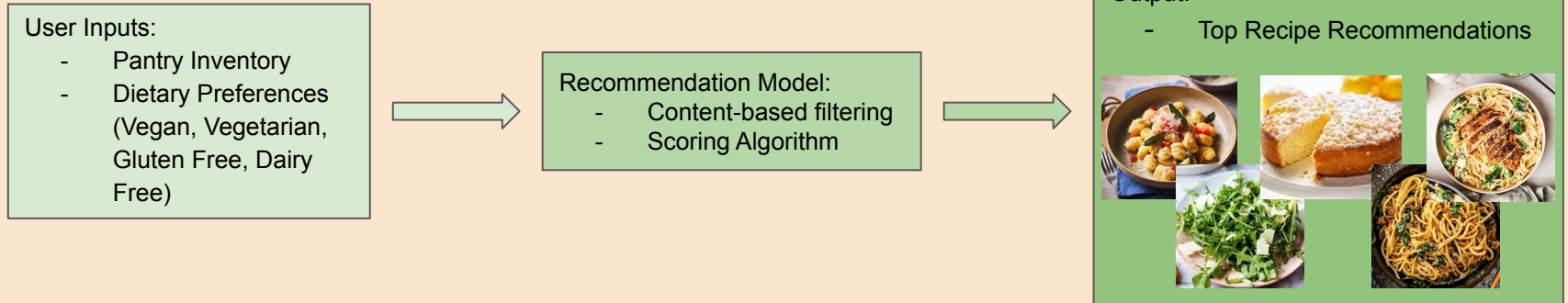
By: Faith, Jared, Kalyani



Smart Nutrition Assistant Companion & Curator (S.N.A.C.C.)

- Machine Learning Model

- Ingredients + Dietary Preference → List of Ranked Recipes
- User experience - Fast, easy, intuitive
- Utility - Efficient, effective



Problem Addressed - Impact of Food Waste on Environment and Economy

- 40% of food is wasted having a direct impact on the environment and consumption of natural resources (rescuingleftovercuisine.org)
- 19% of food waste is from retail, food service, and households (moveforhunger.org)
- Approximately \$243,864,931,645 in food waste per year from America, \$728.14 per capita (epa.gov)



Data Collection and Cleanup

INPUT (Dataset Used):

Extended Recipes Dataset: 64K Dishes

<https://www.kaggle.com/datasets/wafaaelhusseini/extended-recipes-dataset-64k-dishes>

This dataset is an extended and feature-enriched version of the original Recipes Dataset: 64K Dishes (from Kaggle — a high-quality collection of 64,000+ structured recipes).

Transformation Steps:

1. **Data Sanitization:** From Raw CSV to Structured Objects e.g.

"['Salt', 'Pepper!']" → Clean List: ['salt', 'pepper']

Dropping nulls and duplicates prevents any empty recipes from being recommended.

2. **Feature Engineering:** Turning Recipes into Vectors e.g. Ingredients + Cuisine + Taste = Combined_Feature

Useful For:

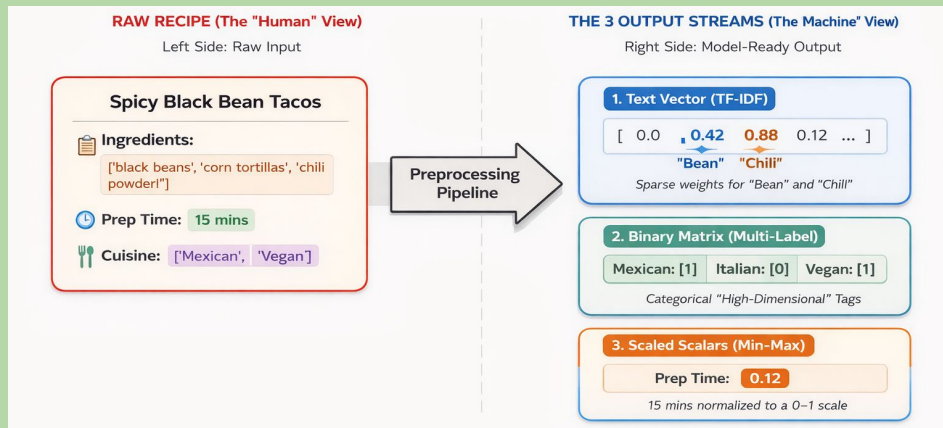
- TF-IDF Vectorization
- Multi-Label Encoding
- Ordinal Mapping

3. Scaling & Integration: Model Readiness & Export Useful For:

- Apply MinMax Scaler
- The training and testing split
- Artifact Export (Saving .pkl files)

OUTPUT (Model-Ready Artifacts):

1. Cleaned Metadata (.csv)
2. The Feature Matrix
3. Fitted Encoders (.pkl)



Model

Data Preprocessing for Recipe Recommendation System

Need for Preprocessing

- Objective: Prepare raw recipe data for machine learning
- Input: recipes_extended.csv
- Output: recipes_cleaned.csv

Issues with Raw Data

- Ingredients stored as strings instead of lists
- Inconsistent text (uppercase, missing values)
- Presence of irrelevant columns

Goal

- Clean and standardize data
- Convert into a machine learning–friendly format

Impact on Model

Before Preprocessing

- Inconsistent data
- Poor model performance

After Preprocessing

- Clean and standardized input
- Improved recommendation accuracy
- Faster processing

Comparison between different models

Model	When It Works Well	Why We Didn't Use It	Verdict
TF-IDF + Cosine Similarity	Text-based data, Similarity matching	Our Chosen Baseline	 Best Fit
K-Means Clustering	Clustering Recipes	Used for Clustering	 Used Alongside
Logistic Regression	Label Prediction	Labels Already Exist	 Not Needed
K-Nearest Neighbors (KNN)	Distance-Based Search	Slow & High Memory Use	 Scales Poorly
Deep Learning (Neural Nets)	Complex Patterns	Too Complex & Costly	 Overkill

How does TF IDF and Cosine Similarity work

Step 1: TF-IDF Vectorization

TF-IDF Matrix Example

Recipes	Chicken	Garlic	Butter	Cake
Recipe A	0.8	0.6	0.1	0.0
Recipe B	0.0	0.5	0.7	0.0
Recipe C	0.0	0.0	0.0	0.9

- Top 5000 Words
- Each Recipe = Vector of Numbers

User Query

chicken garlic butter



[0.8, 0.7, 0.2, 0.0]

Converted to a Vector

Step 2: Cosine Similarity

Compare Vectors



$$\cos \theta = \frac{A \cdot B}{\|A\| \|B\|}$$

Measures Angle Between Vectors

Similarity Score



- Recipe 1: 🍗 Garlic Butter Chicken Pasta Similarity = 0.92 ✓
- Recipe 2: 🍫 Chocolate Cake with Cocoa Similarity = 0.10 ✗

TF-IDF: How Text is Converted to Vector Values

1. Full Form

TF-IDF = **Term Frequency** × **Inverse Document Frequency**

It converts text into a numerical vector, where each dimension = a word.

2. Core Formulas

✔ Term Frequency (TF)

Measures how often a word appears *in a document*.

$$TF(t, d) = \frac{\text{count of term } t \text{ in document } d}{\text{total terms in document}}$$

✔ Inverse Document Frequency (IDF)

Measures how **unique** a word is across *all documents*:

$$IDF(t) = \log \frac{N}{1 + df(t)} \left(\frac{1}{n} \right)$$

$$\mathbf{TF-IDF}(t, d) = TF(t, d) \times IDF(t)$$

3. Example (Recipes)

📧 Documents:

- Doc1: "chicken garlic lemon"
- Doc2: "chicken garlic"
- Doc3: "lemon salad oil"

🔍 Step 2: TF Calculation (for Doc1)

Word	Count	TF
chicken	1	1/3 = 1/3
garlic	1	1/3 = 1/3
lemon	1	1/3 = 1/3
salad	0	0
oil	0	0

➡ Common words → low IDF

➡ Rare words → high IDF

📧 Step 4: TF-IDF Vector

[chicken, garlic, lemon, salad, oil]

🔍 Step 1: Vocabulary

(All unique words)

[chicken, garlic, lemon, salad, oil]

🔍 Step 3: TF Calculation (for Doc1)

Word	df	IDF = $\frac{N}{1} (1+df)$
chicken	2	$\log(3/3) = 0$
garlic	2	$\log(3/3) = 0$
lemon	2	$\log(3/3) = 0$
salad	1	$\log(3/2) > 0$
oil	1	$\log(3/2) > 0$

➡ Common words → low IDF

➡ Rare words → high IDF

→ [0, 0, 0, 1, 1, 0, 0, 0]

Recipe Corpus

Recipe 1: "Bake the chicken with garlic and lemon."

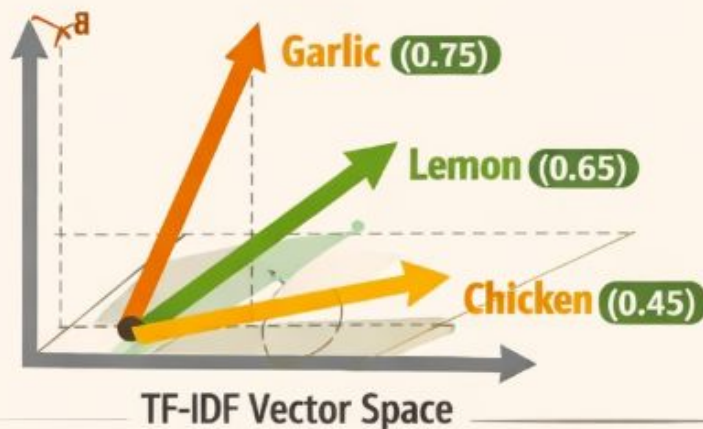
Recipe 2: "Grill the chicken with herbs and garlic."

Recipe 3: "Mix the salad with lemon and olive oil."

Calculate Term Frequency
&
Inverse Document Frequency

Word & Vector Representation

- **garlic:** TF-IDF = 0.75
- **lemon:** TF-IDF = 0.65
- **chicken:** TF-IDF = 0.45
- **olive oil:** TF-IDF = 0.20



High TF-IDF Score:

The word is important in the recipe corpus.

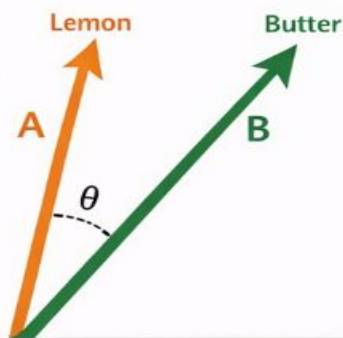
Low TF-IDF Score:

The word is less significant in the recipe corpus.

Cosine Similarity Between Two Recipes

Recipe A: "chicken garlic lemon"

[0.5, 0.5, 0.7, 0]



Recipe B: "chicken garlic butter"

[0.5, 0.5, 0, 0.8]

Vocabulary: [chicken, garlic, lemon, butter]

Dot Product

$$A \cdot B = (0.5 \times 0.5) + (0.5 \times 0.5) + (0.7 \times 0) + (0 \times 0.8) = 0.5$$

Magnitudes

$$|A| = \sqrt{0.99}$$

$$|B| = \sqrt{1.14}$$

Cosine Similarity Calculation

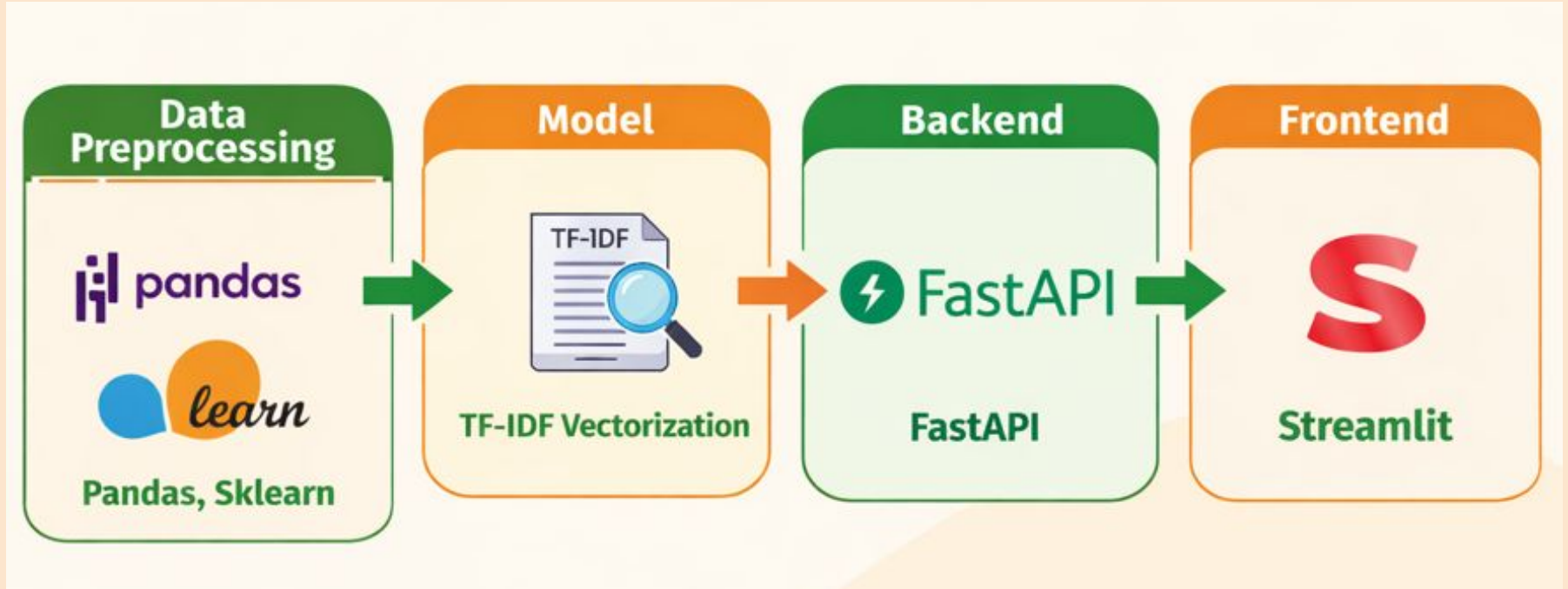
$$\text{Similarity} = \frac{0.5}{\sqrt{0.99} \times \sqrt{1.14}} \approx 0.47$$

✓ Shared: chicken, garlic

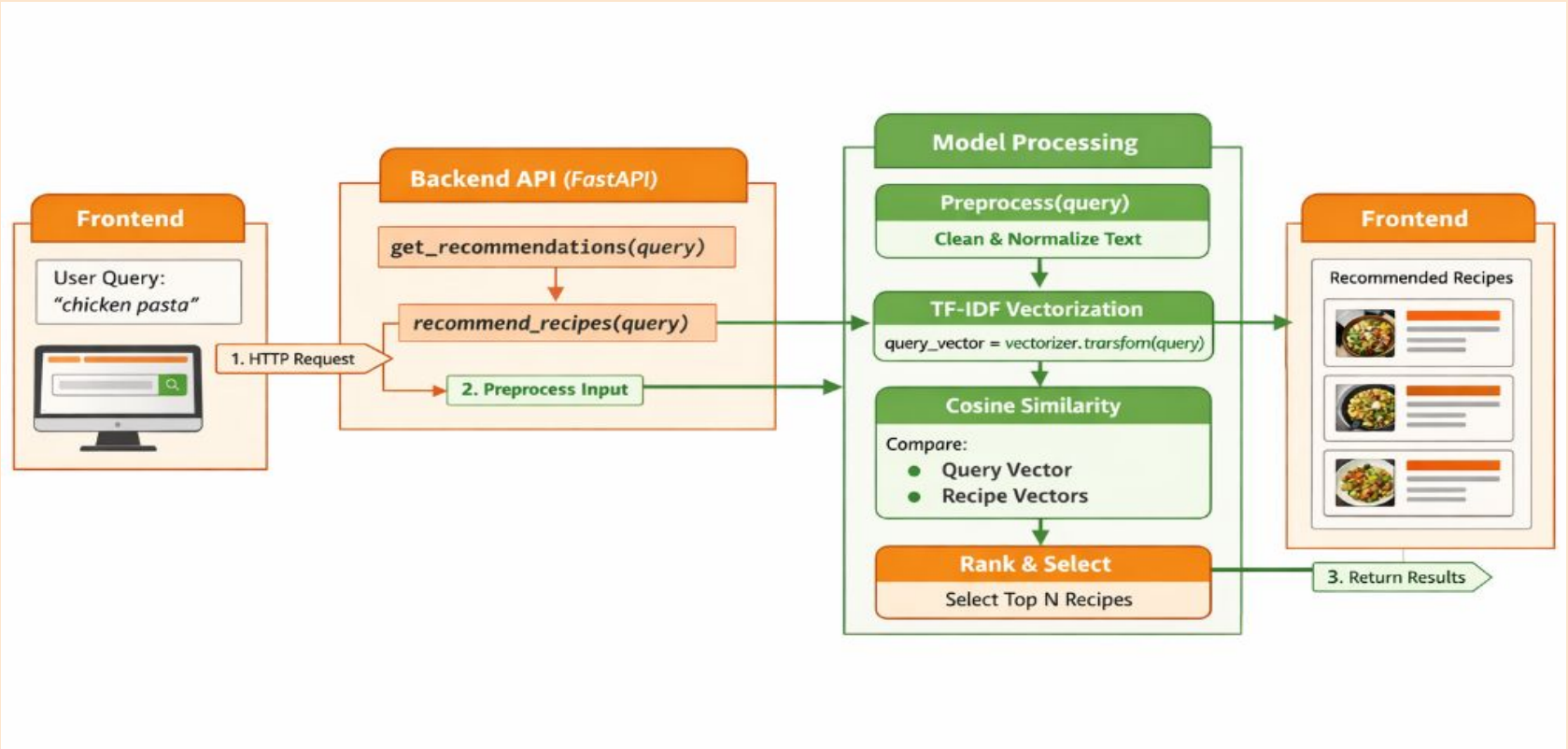
✗ Different: *lemon* vs. *butter*

Measures how similar the ingredient patterns of two recipes are.

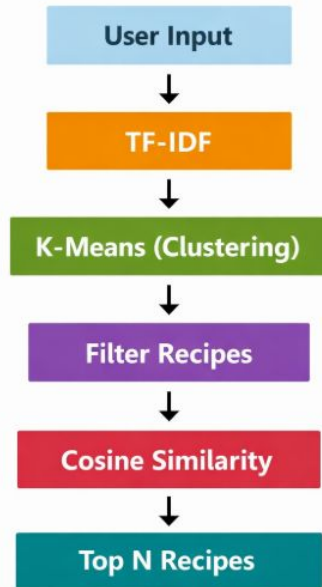
Technology Stack



Project Flow



K-Means Clustering



K-Means clustering is an unsupervised algorithm that groups similar recipes into **K clusters** by assigning each recipe to the nearest centroid and iteratively refining these centers. It speeds up recommendations by narrowing the search to a relevant cluster but is less precise on its own and depends on choosing the right K. In this project, a **hybrid approach** will be used: first, apply K-Means to assign the user query to the closest cluster, then run a similarity model (e.g., cosine similarity on TF-IDF features) within that cluster to rank and return the most relevant recipes—combining speed with accuracy.

Future Workflow: TF-IDF, Cosine Similarity, & K-Means Clustering

Load Data



- Recipes Dataset

- TF-IDF Vectorizer
- TF-IDF Vectorizer

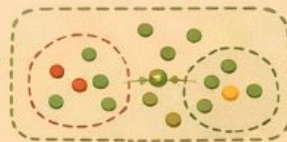
TF-IDF Vectorization

TF-IDF Vectorization



- Convert Text → Numbers
- Features = Top 5000 Words

Dietary Filter



- Features < Top 5000 Words

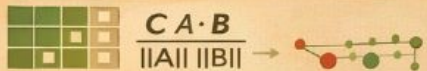
Dietary Filter



- Apply user's dietary preferences

K-Means Clustering

Cluster Prediction



- Groups recipes into Clusters



User Query

e.g., chicken, garlic, butter



Convert to
TF-IDF Vector

e.g., [0.8, 0.7, 0.2, 0.0]



Compare with
Recipes

- Compute Cosine Similarity

Cosine Similarity



K-Means Model



e.g., [0.8, 0.7, 0.2, 0.0]



Recommend Top Recipes

- Return Best Matches
- Use Cluster Recipes

Future Work

- Implementation of K-Means Clustering
 - Improve speed of processing
- Implementation of User Feedback as Input into Model
 - Take user opinion on recipes
 - “Liked it!”, “Disliked it!”, “Hated it!”
 - Feed user preference information as input into the model
 - Can be implemented on a recipe basis
 - Can alternatively be implemented on an ingredient basis

Live demo

Thank You!

Sources

Real World Application

[The Environmental Impact of Food Waste | Move For Hunger](#)

[Estimating the Cost of Food Waste to American Consumers](#)

[Reducing Wasted Food | Rescuing Leftover Cuisine](#)

[U.S. Population \(1950-2025\)](#)

Scikit-learn Documentation

<https://scikit-learn.org/stable/>

TF-IDF Vectorizer

https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.TfidfVectorizer.html

Nearest Neighbors

<https://scikit-learn.org/stable/modules/generated/sklearn.neighbors.NearestNeighbors.html>

Sources

Web References

1. Kaggle Datasets – <https://www.kaggle.com>
2. Scikit-learn Documentation – <https://scikit-learn.org>
3. Streamlit Documentation – <https://docs.streamlit.io>
4. Python Official Site – <https://www.python.org>
5. GitHub – <https://github.com>